

Índices

Álvaro G. Pagliari
alvaro.pagliari@unoesc.edu.br

Roteiro

- Índices
- Exercícios :)





- São estruturas de dados otimizadas que permitem a localização de um determinado dado/tupla mais rapidamente
- A busca em uma tabela se dá de maneira linear $O(n)$, índices devem implementar algoritmos que tornem essa busca mais rápida ($< O(n)$)





- Busque as informações do médico com código = 5;

codm integer	nroa integer	nome character varying(40)	idade smallint	especialidade character(20)	cpf numeric(11,0)	cidade character varying(30)
1	1	Joao	40	ortopedia	10000100000	Florianopolis
2	2	Maria	42	traumatolog	10000110000	Blumenau
3	2	Pedro	51	pediatria	11000100000	Sao Jose
4		Carlos	28	ortopedia	11000110000	Joinville
5	3	Marcia	33	neurologia	11000111000	Biguacu





- Busque as informações do médico com código = 5;

codm integer	nroa integer	nome character varying(40)	idade smallint	especialidade character(20)	cpf numeric(11,0)	cidade character varying(30)
1	1	Joao	40	ortopedia	10000100000	Florianopolis
2	2	Maria	42	traumatolog	10000110000	Blumenau
3	2	Pedro	51	pediatria	11000100000	Sao Jose
4		Carlos	28	ortopedia	11000110000	Joinville
5	3	Marcia	33	neurologia	11000111000	Biguacu





- Busque as informações do médico com código = 5;

codm integer	nroa integer	nome character varying(40)	idade smallint	especialidade character(20)	cpf numeric(11,0)	cidade character varying(30)
1	1	Joao	40	ortopedia	10000100000	Florianopolis
2	2	Maria	42	traumatolog	10000110000	Blumenau
3	2	Pedro	51	pediatria	11000100000	Sao Jose
4		Carlos	28	ortopedia	11000110000	Joinville
5	3	Marcia	33	neurologia	11000111000	Biguacu





- Busque as informações do médico com código = 5;

codm integer	nroa integer	nome character varying(40)	idade smallint	especialidade character(20)	cpf numeric(11,0)	cidade character varying(30)
1	1	Joao	40	ortopedia	10000100000	Florianopolis
2	2	Maria	42	traumatolog	10000110000	Blumenau
3	2	Pedro	51	pediatria	11000100000	Sao Jose
4		Carlos	28	ortopedia	11000110000	Joinville
5	3	Marcia	33	neurologia	11000111000	Biguacu





- Busque as informações do médico com código = 5;

codm integer	nroa integer	nome character varying(40)	idade smallint	especialidade character(20)	cpf numeric(11,0)	cidade character varying(30)
1	1	Joao	40	ortopedia	10000100000	Florianopolis
2	2	Maria	42	traumatolog	10000110000	Blumenau
3	2	Pedro	51	pediatria	11000100000	Sao Jose
4		Carlos	28	ortopedia	11000110000	Joinvile
5	3	Marcia	33	neurologia	11000111000	Biguacu





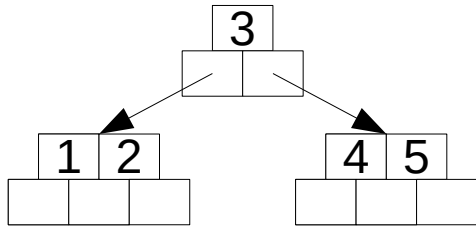
- Busque as informações do médico com código = 5;

codm integer	nroa integer	nome character varying(40)	idade smallint	especialidade character(20)	cpf numeric(11,0)	cidade character varying(30)
1	1	Joao	40	ortopedia	10000100000	Florianopolis
2	2	Maria	42	traumatolog	10000110000	Blumenau
3	2	Pedro	51	pediatria	11000100000	Sao Jose
4		Carlos	28	ortopedia	11000110000	Joinville
5	3	Marcia	33	neurologia	11000111000	Biguacu



Índices

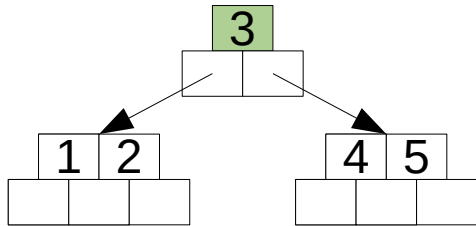
- Busque as informações do médico com código = 5;
 - Com índice



codm	nroa	nome	idade	especialidade	cpf	cidade
integer	integer	character varying(40)	smallint	character(20)	numeric(11,0)	character varying(30)
1	1	Joao	40	ortopedia	10000100000	Florianopolis
2	2	Maria	42	traumatolog	10000110000	Blumenau
3	2	Pedro	51	pediatria	11000100000	Sao Jose
4		Carlos	28	ortopedia	11000110000	Joinvile
5	3	Marcia	33	neurologia	11000111000	Biguacu

Índices

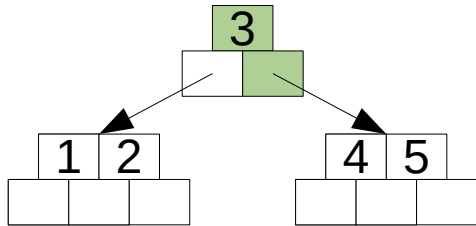
- Busque as informações do médico com código = 5;
 - Com índice



codm	nroa	nome	idade	especialidade	cpf	cidade
integer	integer	character varying(40)	smallint	character(20)	numeric(11,0)	character varying(30)
1	1	Joao	40	ortopedia	10000100000	Florianopolis
2	2	Maria	42	traumatolog	10000110000	Blumenau
3	2	Pedro	51	pediatria	11000100000	Sao Jose
4		Carlos	28	ortopedia	11000110000	Joinvile
5	3	Marcia	33	neurologia	11000111000	Biguacu

Índices

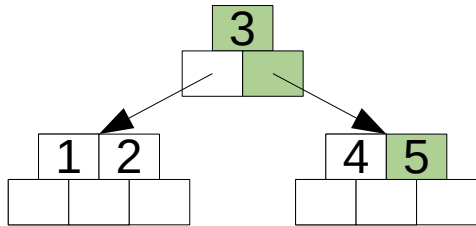
- Busque as informações do médico com código = 5;
 - Com índice



codm	nroa	nome	idade	especialidade	cpf	cidade
integer	integer	character varying(40)	smallint	character(20)	numeric(11,0)	character varying(30)
1	1	Joao	40	ortopedia	10000100000	Florianopolis
2	2	Maria	42	traumatolog	10000110000	Blumenau
3	2	Pedro	51	pediatria	11000100000	Sao Jose
4		Carlos	28	ortopedia	11000110000	Joinvile
5	3	Marcia	33	neurologia	11000111000	Biguacu

Índices

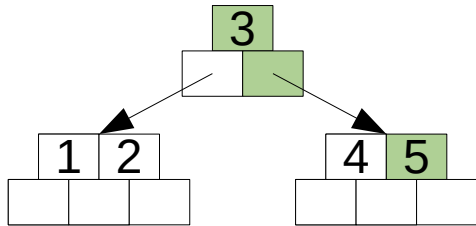
- Busque as informações do médico com código = 5;
 - Com índice



codm	nroa	nome	idade	especialidade	cpf	cidade
integer	integer	character varying(40)	smallint	character(20)	numeric(11,0)	character varying(30)
1	1	Joao	40	ortopedia	10000100000	Florianopolis
2	2	Maria	42	traumatolog	10000110000	Blumenau
3	2	Pedro	51	pediatria	11000100000	Sao Jose
4		Carlos	28	ortopedia	11000110000	Joinvile
5	3	Marcia	33	neurologia	11000111000	Biguacu

Índices

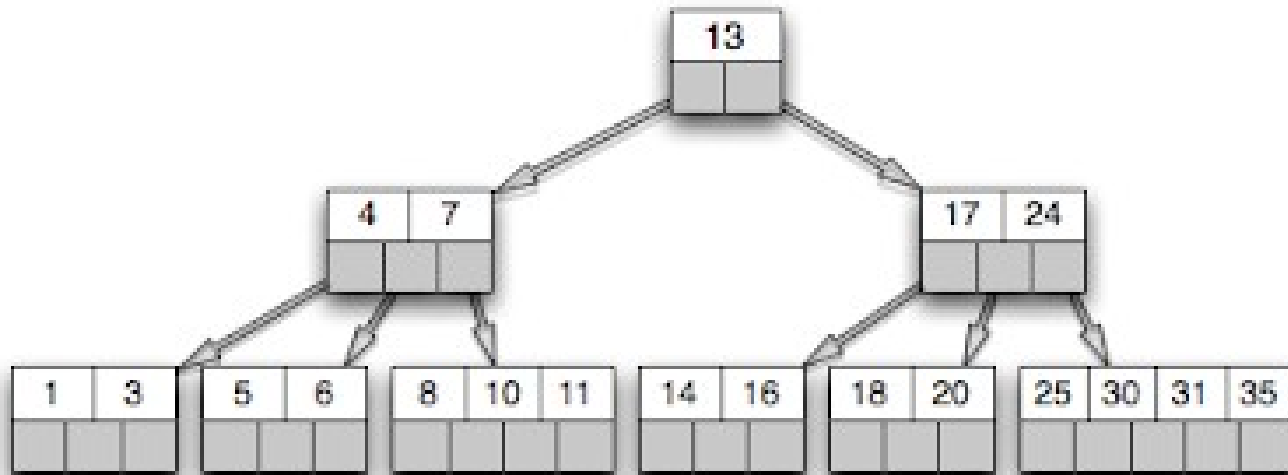
- Busque as informações do médico com código = 5;
 - Com índice



codm	nroa	nome	idade	especialidade	cpf	cidade
integer	integer	character varying(40)	smallint	character(20)	numeric(11,0)	character varying(30)
1	1	Joao	40	ortopedia	10000100000	Florianopolis
2	2	Maria	42	traumatolog	10000110000	Blumenau
3	2	Pedro	51	pediatria	11000100000	Sao Jose
4		Carlos	28	ortopedia	11000110000	Joinvile
5	3	Marcia	33	neurologia	11000111000	Biguacu

Índices

- Busque as informações do médico com código = 31;
 - Com índice
 - Imagine um cenário maior



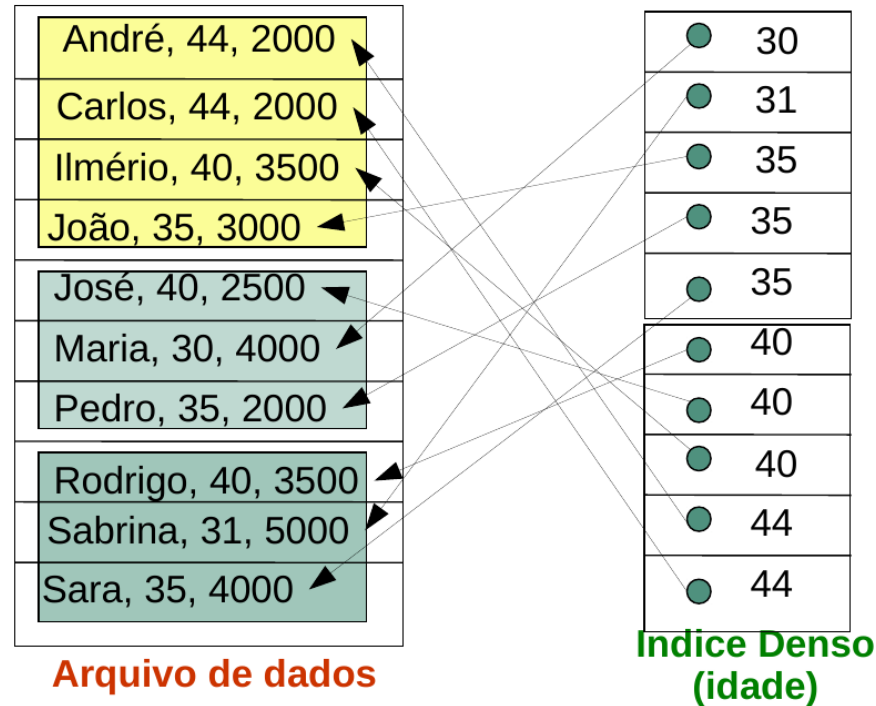


- Para criar um índice devem ser considerados alguns aspectos:
 - Chave que irá compor
 - Chave primária, de ordenação ou mesmo chave de pesquisa
 - É criado um novo documento apenas contendo os dados da chave e o ponteiro do dado
 - local pode ser a localização exata da tupla ou pode ser o cluster ou o bloco
 - Os índices podem ser densos ou esparsos



Índices

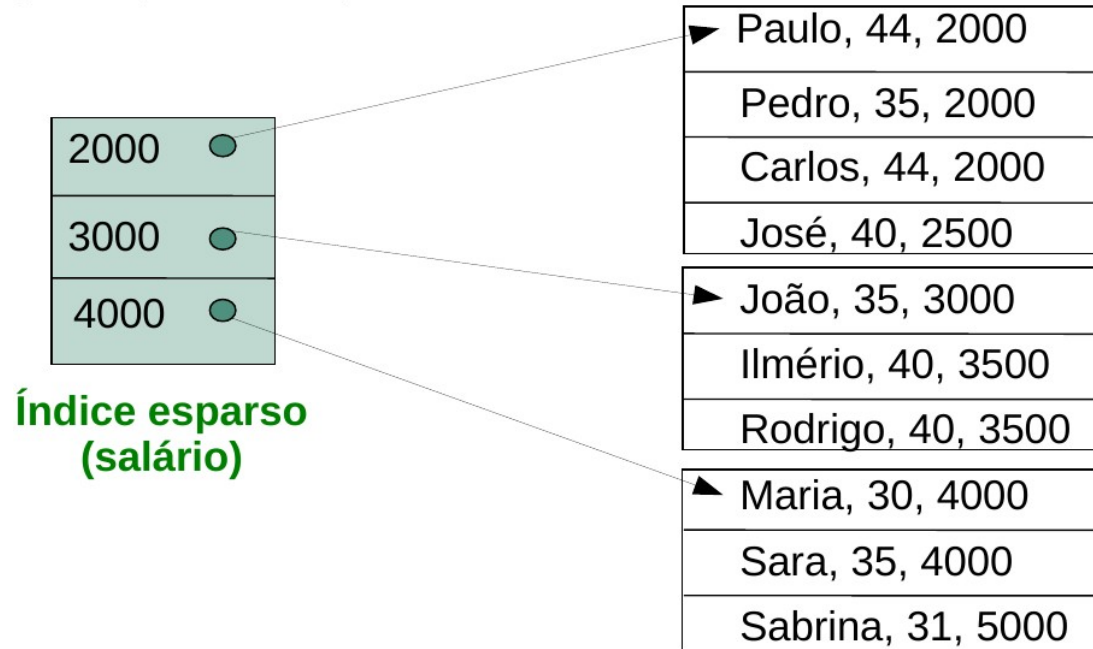
- Índices densos possuem entrada para todas as chaves de dados



- Índices densos possuem entrada para todas as chaves de dados
 - Vantagens (além de otimizar a consulta)
 - O número de blocos para armazenar o índice é, geralmente, menor que para armazenar os dados
 - Pode-se utilizar a busca binária para buscar um Registro
 - O índice pode caber na memória principal (buffer), diminuindo o número de I/O's em uma busca



- Índices esparsos possuem chaves que apontam blocos de dados





- Índices esparsos possuem chaves que apontam blocos de dados
 - Característica
 - São menores que os índices esparsos
 - Faz um scan no bloco de dados
 - A busca é feita no índice esparsos valor $\geq$ key
 - O arquivo de dados deve estar ordenado pela chave de busca





- Tipos de índice
 - Primário
 - A chave do índice é composta pela chave primária da tabela
 - A maioria dos SGBD cria índices primários automaticamente
 - Não permitem duplicatas
 - Podem ser agrupados
 - Secundário
 - Outras colunas da tabela participam
 - Permitem duplicatas





- Tipos de índice
 - Simples
 - Envolve apenas uma coluna
 - Composto
 - Envolve várias colunas
- Algoritmos (Depende de cada SGBD)
 - **B-Tree**
 - R-Tree
 - Gist
 - **Hash**



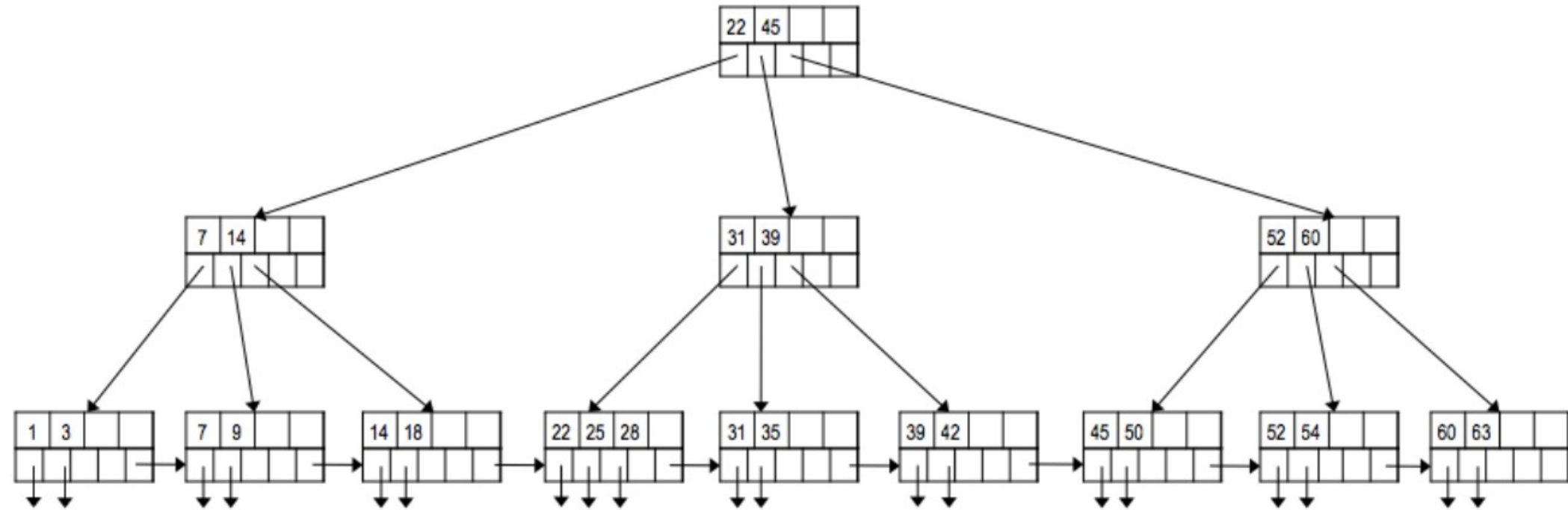


- **B-Tree**

- É o índice padrão para a maioria dos BDs
- Utilizado para operações com qualquer operador matemático: $=$, $<>$, $>$, $<$, $>=$, $<=$
- A B-Tree implementada pelo PostgreSQL apenas armazena os ponteiros nos nodos folha.
- As folhas armazenam os dados e são mantidas como uma lista encadeada



- **B-Tree**



- **B-Tree**

- O tamanho da árvore dita o número mínimo de operações
- Qual a altura máxima de uma árvore B com m chaves?
 - Sendo N o maior número de chaves na árvore

$$h = \log_{\frac{m}{2}} \left(\frac{N+1}{2} \right)$$



- **B-Tree**

- Dada um árvores B+ de ordem b , os tempos serão:
 - Inserção
 - $O(\log_b(n))$
 - *Busca*
 - $O(\log_b(n))$
 - Sendo n o número de chaves





- **B-Tree**

- Mas como funciona isso?
 - Let's play
 - <https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html>





- Tipos de índice
 - Simples
 - Envolve apenas uma coluna
 - Composto
 - Envolve várias colunas
- Algoritmos (Depende de cada SGBD)
 - B-Tree
 - R-Tree
 - Gist
 - **Hash**





- **Hash**

- Possui um desempenho elevado, a busca usualmente é $O(1)$.
- Apenas pode ser utilizado em buscas por igualdade =
- Já utilizaram algum tipo de hash?





- **Hash**

- Possui um desempenho elevado, a busca usualmente é $O(1)$.
- Apenas pode ser utilizado em buscas por igualdade =
- Já utilizaram algum tipo de hash?
- Uma tabela de dispersão ou tabela de hash (*hash table*) é um vetor cada uma de cujas posições armazena zero, uma, ou mais chaves (e valores associados).





- **Hash**

- Tabela de hash ou bucket
 - Uma tabela de hash armazena os dados em buckets. Bucket é considerado uma unidade de armazenamento.
 - Normalmente armazena um bloco de disco completo, que por sua vez pode armazenar um ou mais registros.
- Função de Hash ou função de dispersão
 - A função hash mapeia o conjunto de chaves de pesquisa para o endereço onde os registros reais são armazenados.
 - De maneira geral é uma chave de acesso para cada bucket





- Como criar um índice

CREATE [UNIQUE] INDEX nome **ON** tabela (atributo₁, atributo_n)

- Ex

CREATE INDEX esp_med **ON** medicos (especialidade)

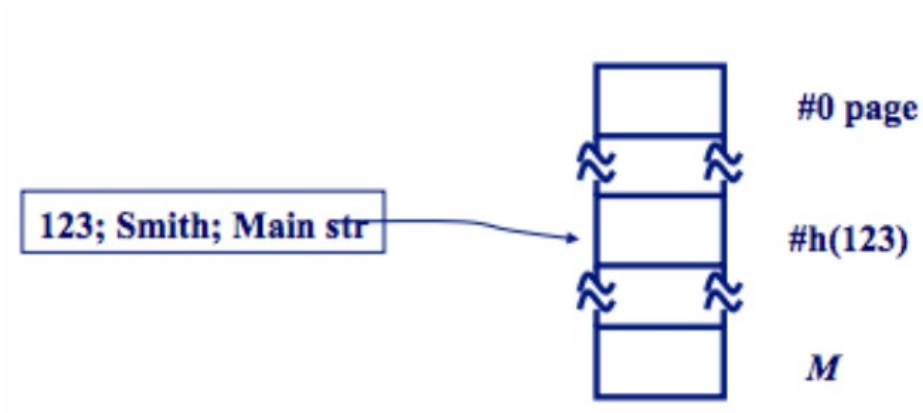
- Excluir índice

DROP INDEX nome;



- **Hash Estático**

- De maneira geral transforma todas as chaves n em um conjunto m de buckets
- Utiliza uma função do tipo $hash(key)$
- Podem ser clusterizados ou não-clusterizados





- **Hash Estático**

- Inserção
 - Aplica a função de hash sobre a chave e identifica o endereço de armazenamento
- Busca
 - Aplica a função de hash sobre a chave de busca e identifica o endereço do ponteiro
- Exclusão
 - Realiza um operação de busca e depois apaga a entrada





- **Hash Estático**

- No pior caso todas as chaves são mapeadas para os mesmos endereços
- No melhor caso cada chave possui um valor distinto de endereço
- Como definir uma função de hash
 - Distribuição e Aleatoriedade
- Escalável?



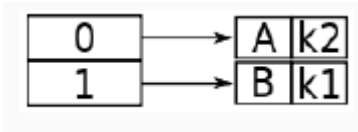


- **Hash Dinâmico**

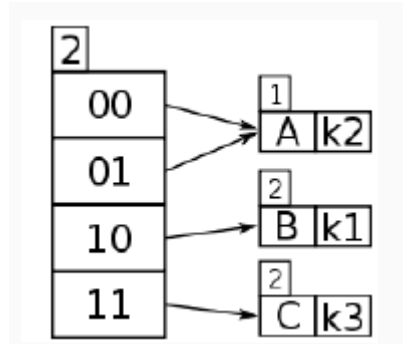
- A função é modificada em tempo de execução, se expandindo e retraindo
- Hash Extensível
 - Calcula as chaves com base em um conjunto de bits, conforme o número de chaves cresce o número de bits aumenta



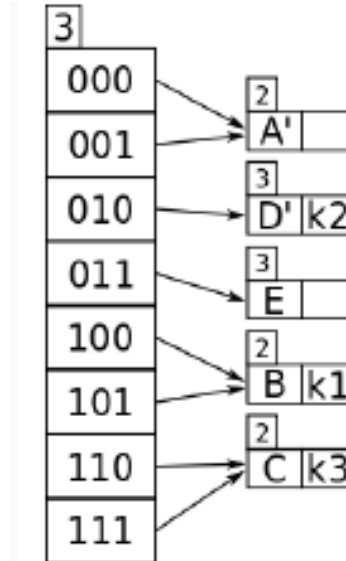
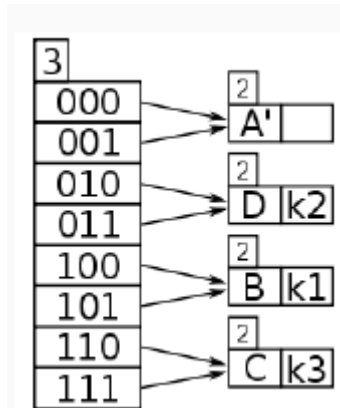
- **Hash Dinâmico**



- **Hash Dinâmico**



- Hash Dinâmico





- **Hash Dinâmico**

- Inserção
 - Calcula o hash da chave e verifica se necessita expandir
 - Se expandir adiciona mais um bit ao hash
- Busca
 - Utiliza os primeiros bits para busca otimizada
- Exclusão
 - Busca seguida da remoção do dado





- **Hash**
 - E se duas chaves distintas obterem o mesmo valor de endereço?



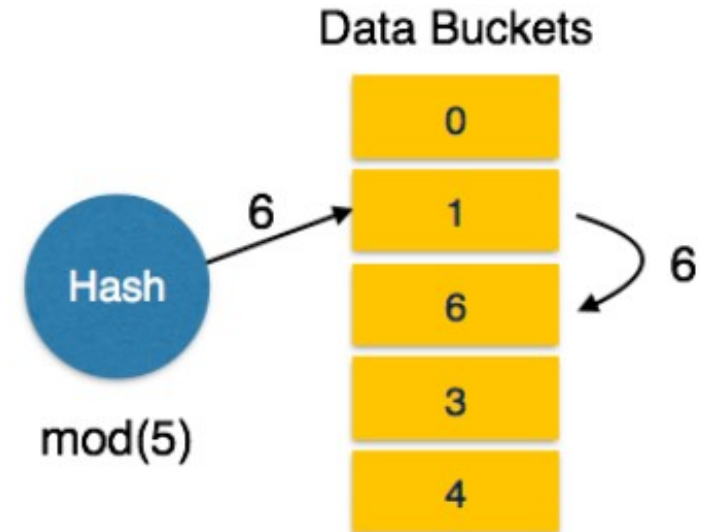


- **Hash**
 - Colisões
 - Acontece quando duas chaves distintas obtêm o mesmo endereço
 - Como tratar
 - Recriar o hash (rehashing)
 - Colocar o valor no próximo endereço



- **Hash**

- Colisões
 - Acontece quando duas chaves distintas obtêm o mesmo endereço
- Como tratar
 - Recriar o hash (rehashing)
 - Colocar o valor no próximo endereço





- **Hash**

- Colisões
 - Acontece quando duas chaves distintas obtêm o mesmo endereço
- Como tratar
 - Recriar o hash (*rehashing*)
 - Colocar o valor no próximo endereço
 - Separação em canais (*overflow chaining*)



- Hash

- Colisão

- Acesso
 - Memória

- Comparação

- Registro
 - Contagem

- Separação em canais (*overflow chaining*)



em o



- Como criar um índice
CREATE [UNIQUE] INDEX nome **ON** tabela (atributo₁,
atributo_n)
- Ex
CREATE INDEX esp_med **ON** medicos (especialidade)
- Excluir índice
DROP INDEX nome;



Índices

- Como criar um índice

```
CREATE [ UNIQUE ] INDEX [ CONCURRENTLY ] [ IF NOT EXISTS ] nome_indice ON [ ONLY ]  
tabela [ USING method ] ( { column | ( expression ) }  
    [ ASC | DESC ] [ NULLS { FIRST | LAST } ] [ , ... ] )  
    [ INCLUDE ( column_name [ , ... ] ) ]  
    [ NULLS [ NOT ] DISTINCT ]  
    [ WITH ( storage_parameter [= value] [ , ... ] ) ]  
    [ TABLESPACE tablespace_name ]  
    [ WHERE predicate ]
```

- Ex

```
CREATE INDEX esp_med ON medicos USING hash (especialidade)  
CREATE INDEX nro_med ON medicos USING btree (nroa ASC NULLS FIRST)
```

- Excluir índice

```
DROP INDEX nome;
```

Índices

- UNIQUE: não permite duplicação
- CONCURRENTLY: permite criar um índice sem criar locks na tabela
- IF NOT EXISTS: somente cria se o índice não existir
- INCLUDE: permite incluir colunas não-indexadas ao índice, evitando a leitura da tabela, porém tem um custo alto
- ONLY: evita criação de índice em tabelas particionadas
- NULLS [NOT] DISTINCT: indica se em um índice único os valores NULLs são considerados distintos ou não. Por padrão são distintos, ou seja, o índice pode conter diversos valores NULL repetidos



Questão Enade

(ENADE 2008 – BACHARELADO): Considere a relação EMPREGADO (NumeroEmp, RG, nome, sobrenome, salário, endereço), em que o atributo grifado corresponde à chave primária da relação. Suponha que se deseje realizar as seguintes consultas:

1. Listar o nome dos empregados com sobrenome Silva;
2. Listar o nome dos empregados em ordem crescente de seus sobrenomes.

Em relação à definição de um índice sobre o atributo sobrenome para melhorar o desempenho das consultas acima, julgue os itens a seguir.

- I. Um índice que implemente Árvore-B+ será adequado para melhorar o desempenho da consulta 1.
- II. Um índice que implemente Árvore-B+ será adequado para melhorar o desempenho da consulta 2.
- III. Um índice que implemente uma função hash será adequado para melhorar o desempenho da consulta 1.
- IV. Um índice que implemente uma função hash será adequado para melhorar o desempenho da consulta 2.

Exercícios

- 1) Crie um índice que otimize a busca do CPF de um cliente com base na placa de seu veículo.
- 2) Crie um índice que otimize consultas na tabela reservas que busquem o número do quarto que não possui reservas após uma determinada data.
- 3) Crie um índice que otimize consultas que listem em ordem crescente os tipos de quartos por preço.

